FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System

VIVA PREPARATION

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	VIVA PREPARATION
Version	v1.0.0
Date	02 July 2026

University Final Year Project Viva — Questions and Ideal Answers

Project Introduction Questions

Q: What is the title of your project?

"Smart Medicine Availability and Intelligent Janaushadhi Recommendation System."

It is a web application that helps patients find affordable generic medicines, discover Jan Aushadhi alternatives for branded drugs, and locate nearby Jan Aushadhi pharmacies on an interactive map.

Q: What problem does your project solve?

In India, branded medicines are often 5–10 times more expensive than their generic equivalents. Most patients are unaware that:

1. Cheaper generic medicines with the same active composition exist
2. Over 9,000 Jan Aushadhi stores across India sell government-approved affordable generics

Our system bridges this gap — a patient searches for a branded medicine, gets shown the generic equivalent with price comparison, and sees the nearest pharmacy where they can buy it.

Q: Who are the users of your system?

Four types of users:

1. **Patients/Doctors** — search medicines, find generics, locate pharmacies
2. **Pharmacists** — manage medicine inventory
3. **Admins** — manage users, medicines, pharmacies, view analytics

---

Q: What technology stack did you use for the frontend?

- **React 19** — UI library
- **Vite** — build tool
- **Tailwind CSS v4** — styling
- **React Router DOM v7** — client-side routing
- **TanStack React Query** — server-state caching
- **Axios** — HTTP client
- **React Hook Form + Zod** — form management and validation
- **React Leaflet** — interactive maps
- **React Toastify** — notifications

Q: Why did you choose React?

Three reasons:

1. **Component reusability** — build once, use everywhere. This project has 50+ reusable components.
2. **Industry standard** — React is the most widely used frontend framework. Relevant for placements.
3. **Ecosystem** — React Query, React Hook Form, React Leaflet — excellent libraries for every need.

Technical Questions — Architecture

Q: Explain your project folder structure.

(Refer to PROJECT_FOLDER_STRUCTURE.md)

The project follows feature-based + layer-based hybrid architecture:

- `pages/` — screens corresponding to routes
- `components/` — reusable UI blocks (ui, forms, cards, navigation, etc.)
- `layouts/` — persistent shells (Navbar+Footer, Sidebar+TopBar)
- `contexts/` — global state (Auth, Theme)
- `services/` — API calls (all HTTP requests)
- `hooks/` — reusable stateful logic
- `utils/` — pure utility functions
- `constants/` — no magic strings

This architecture is scalable because adding a new feature = adding one page file + one service function. No existing code changes.

Q: What are the five layouts and when is each used?

Layout	Used For
MainLayout	Public home page — has Navbar + Footer
AuthLayout	Login/Register/OTP — centered card, no navigation
UserLayout	Patient dashboard — Sidebar + TopBar
PharmacyLayout	Pharmacist pages — Sidebar + TopBar
AdminLayout	Admin portal — Sidebar + TopBar

Each layout uses `<Outlet />` from React Router — the page content renders inside the layout.

Q: What is a Protected Route and how did you implement it?

A Protected Route is a route wrapper that checks authentication before rendering the page. If the user is not logged in, they are redirected to `/login`. If their role is wrong, they are redirected to `/unauthorized`.

```
function ProtectedRoute({ allowedRoles }) {
  const { isAuthenticated, currentUser } = useAuth()

  if (!isAuthenticated) return <Navigate to="/login" />
  if (!allowedRoles.includes(currentUser.role)) return <Navigate to="/unauthorized" />

  return <Outlet />
}
```

Q: What is lazy loading and how did it help your application?

Lazy loading splits the JavaScript bundle — pages only load their code when the user navigates to them. Implemented with `React.lazy()` and dynamic `import()`.

Impact: Without lazy loading: one 963KB bundle. With lazy loading: 464KB main bundle + 43 smaller chunks. 50% faster initial page load.

Q: How does authentication work in your frontend?

1. User submits login form
2. `AuthContext.login()` is called — calls `authService.login()` (when backend ready)
3. Backend returns `{ user, accessToken, refreshToken }`
4. Tokens stored in localStorage via `storage.js`
5. `currentUser` state updated in `AuthContext`
6. Every subsequent API call automatically gets the JWT token (via `axiosClient.js` request interceptor)
7. When token expires → 401 response → interceptor clears session → redirect to `/session-expired`

Q: What is Context API and why did you use it?

Context API provides global state without prop drilling. In this project:

- `AuthContext` provides authentication state to every component
- `ThemeContext` provides the current theme
- Any component can call `useAuth()` to get `currentUser` and `logout()` — no props needed

Alternative would be Redux, but Context is sufficient for this project's scale.

Technical Questions — Components

Q: What is a reusable component? Give an example from your project.

A reusable component is a UI element written once and used in many places, configurable via props.

Example — `Badge` component:

Used in: SearchResultCard (In Stock), UserDashboard (role badge), InventoryPage (low stock), AdminUsers (role), AdminMedicines (status).

Without it: 6 different teams would create 6 different "badge" HTML snippets with 6 different styles.

Q: Explain the SearchResultCard component.

`SearchResultCard` is the richest component in the system. It displays:

- Medicine name, generic name, composition, manufacturer
- Smart availability badges (In Stock, Out of Stock, Limited)
- Feature badges (Generic, Jan Aushadhi, Affordable, New)
- Price with MRP strikethrough and savings percentage
- Compare toggle (add to comparison up to 4)
- Save/bookmark (local state)
- Share (Web Share API)
- "Best Value" ribbon for the top recommendation

It supports two layouts: `grid` (vertical) and `list` (horizontal).

Q: What is AppErrorBoundary and why is it a class component?

AppErrorBoundary catches JavaScript errors anywhere in the component tree and shows `ServerErrorPage` instead of crashing.

It must be a class component because it uses `componentDidCatch` lifecycle method — there is no hooks equivalent for error boundaries in React.

Q: How does your form validation work?

Three layers:

1. **Zod schema** (`authSchemas.js`) — defines validation rules
2. **React Hook Form** — manages form state and calls validation
3. `@hookform/resolvers` — connects Zod to React Hook Form

```
const { register, handleSubmit, formState: { errors } } = useForm({
  resolver: zodResolver(loginSchema)
})
```

When the user submits, Zod validates the data. Errors appear under each field automatically.

Technical Questions — API & Backend

Q: How will your frontend connect to the FastAPI backend?

The connection is already built. Only three steps needed:

1. Set `VITE_API_BASE_URL=http://localhost:8000/api/v1` in `.env`
2. In `AuthContext.jsx` — remove mock data, uncomment `await authService.login(credentials)`
3. In each page — replace mock data with `medicineService.getById(id)` calls using React Query

The service files, Axios client, JWT interceptors, error handling — all ready.

Q: What is Axios and why did you use it?

Axios is an HTTP client that wraps the browser `fetch` API with:

- Automatic JSON serialisation/deserialisation
- Request/response interceptors (JWT, error handling)
- Timeout support
- Consistent error objects

In this project, the `axiosClient.js` is a shared instance with the API base URL and JWT interceptors pre-configured.

Q: What does the Axios response interceptor do?

It intercepts every response. On 401 Unauthorized:

1. Clears `accessToken`, `refreshToken`, `user` from localStorage
2. Redirects to `/session-expired`

This ensures expired sessions are handled gracefully without requiring every API call to check for 401.

Questions About The Healthcare Domain

Q: What is Jan Aushadhi?

The **Pradhan Mantri Bhartiya Janaushadhi Pariyojana (PMBJP)** is a Government of India initiative that establishes Jan Aushadhi stores selling generic medicines at affordable prices. Currently over 9,000 stores operate across India. Generic medicines can be 50–90% cheaper than branded equivalents.

Q: What is the difference between a branded medicine and a generic medicine?

A **branded medicine** is manufactured by the original pharmaceutical company under a brand name (e.g., "Crocin" by GlaxoSmithKline).

A **generic medicine** contains the same active ingredient(s), same dosage, same form, and is therapeutically equivalent (e.g., "Paracetamol 500mg" — the molecule is identical).

The price difference exists because branded medicines include R&D recovery costs. Generics are manufactured after the patent expires.

Q: What security considerations did you implement?

Frontend security implemented:

1. **JWT Bearer tokens** for all API requests
2. **Role-based access control** — `ProtectedRoute` checks roles
3. **Automatic session expiry** on 401 response
4. **Protected routes** — unauthenticated users cannot access dashboards
5. **Form validation** — prevents invalid data submission
6. **No secrets in frontend code** — all sensitive config in `.env`

Not implemented (frontend-only):

• XSS protection: handled by React's JSX escaping by default
• CSRF: handled by FastAPI backend

Q: What accessibility features did you implement?

1. `<nav aria-label="Breadcrumb">` + `aria-current="page"` on Breadcrumb
2. All icon buttons have `aria-label`
3. All decorative icons have `aria-hidden="true"`
4. Skip-to-content link in `index.html`
5. `focus-visible:ring` focus indicators on all interactive elements
6. Semantic HTML (`<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`)
7. Form fields connected with `aria-describedby` for errors
8. `<noscript>` fallback for JS-disabled browsers

Q: What future enhancements can be made?

(From FUTURE_ENHANCEMENTS.md)

1. **Barcode Scanner** — scan medicine barcode → instant details

2. **Voice Search** — speak medicine name → search
3. **OCR** — photograph prescription → extract medicines
4. **Medicine Reminders** — push notifications for dose schedule
5. **AI Recommendation** — ML model for personalised suggestions
6. **PWA** — installable app, offline mode

Examiner Curveball Questions

Q: Why React and not Angular or Vue?

React uses a component-based model with JSX and is the industry's most popular frontend library. Angular is opinionated and heavier — suitable for enterprise projects. Vue is excellent but has smaller Indian industry adoption. React was chosen for its ecosystem, community, and placement relevance.

Q: How would you add a new page to this project?

1. Create `src/pages/newFeature/NewPage.jsx`
2. Create service function if needed in `src/services/`
3. Add route in `src/routes/AppRouter.jsx`
4. Add route constant in `src/constants/routes.js`
5. Add nav item in `src/constants/navConfig.js`

No existing files need to change beyond these additions.

Q: What would you do differently if you rebuilt this project?

Good answer to give:

• Add TypeScript from the start — better type safety during development
• Add proper unit tests (Vitest + React Testing Library) from Module 1
• Consider using Zustand for state management once the app exceeds 3 contexts
• These are improvements for production scale, not mistakes in the current architecture